NovaTech Industries — Checkpoint 2.9

Integrated Supplier Evaluation Engine

Project: Intelligent Procurement & Logistics Management Platform

Dataset: Synthetic v2

Case-study reporting date: 2026-10-21

Status: Learning prototype; no purchasing authorization

1. Learning objectives

- Join three analytical CSV outputs to supplier, product and requisition reference data.

- Distinguish preliminary quotation checks, compliance checks and evidence sufficiency.

- Normalize cost and lead-time scores within each requisition's eligible quotation set.

- Calculate a weighted score without overriding procurement controls.

- Produce a reproducible audit-friendly CSV.

2. Inputs and schema

Input file	Required fields
`data/analysis/v2/quotation_eligibility_results.csv`	quotation_id, requisition_id, supplier_id, product_id, total_cost_eur, preliminary_status, reasons
`data/analysis/v2/supplier_performance_metrics.csv`	supplier_id, quality_score, reliability_score, evidence_status, report_as_of
`data/analysis/v2/supplier_category_compliance.csv`	supplier_id, category, compliance_status, report_as_of
`data/synthetic/v2/products.csv`	product_id, category
`data/synthetic/v2/quotation_items.csv`	quotation_id, lead_time_days
`data/synthetic/v2/purchase_requisitions.csv`	requisition_id, request_date

3. Evaluation rules

1. Preserve any previous `BLOCKED` result (expired quotation, inactive supplier, insufficient stock, late delivery, etc.).

2. Require category-specific compliance status `VALID`; otherwise block the quotation.

3. Require `EVIDENCE_SUFFICIENT` to calculate quality and reliability scores. Otherwise mark the quotation `REVIEW_REQUIRED` (unless another rule blocks it).

4. Validate positive cost and lead time, and quality/reliability scores in the 0–100 range.

5. Only quotations marked `ELIGIBLE_FOR_ANALYSIS` enter the normalization pool **for their own requisition**.

6. Score the eligible pool, but do not issue purchase orders or approve a supplier automatically.

Formulae:

- Cost score = `100 × minimum eligible total cost / quotation total cost`

- Delivery score = `100 × shortest eligible lead time / quotation lead time`

- Weighted score = `0.40 × cost + 0.25 × delivery + 0.20 × quality + 0.15 × reliability`

When a requisition has only one eligible quotation, its relative cost and delivery scores are both 100; this is **not** proof that the offer is market-competitive. Procurement review is still required.

4. Date limitation

The requisitions are dated **2026-10-15**, while the performance/compliance reporting snapshots are dated **2026-10-21**. Therefore, the prototype performs a **retrospective demonstration**, not a historical point-in-time approval. For production, store effective-dated compliance and performance history, and evaluate using information available at the decision time. Do not treat the 2026-10-21 snapshot as evidence that a supplier was compliant on 2026-10-15.

5. Installation and execution

Place the Python script at:

`src/analysis/evaluate_suppliers_v2.py`

From the project root:

```
python3 src/analysis/evaluate_suppliers_v2.py
```

The output is:

`data/analysis/v2/supplier_evaluation_results.csv`

The script uses the Python standard library only. It reads existing CSVs and writes a new analysis CSV; the synthetic source files are not modified.

6. Tested reference outcome

With the deterministic v2 generator scripts used in earlier checkpoints:

```
Created data/analysis/v2/supplier_evaluation_results.csv: 14 records
Eligible for analysis: 10
Manual review required: 0
Blocked: 4
Top-ranked (not approved): 5
```

These counts are a **reference test**, not a guarantee for independently modified CSVs. The generator includes a low-history supplier, but that supplier may not have a quotation in the 14-row quotation sample. Therefore zero quotation-level `REVIEW_REQUIRED` rows can be valid even though the supplier metrics table has one supplier requiring review.

7. Validation exercise

Run:

```
python3 - <<'PY'
import csv
from pathlib import Path
p = Path('data/analysis/v2/supplier_evaluation_results.csv')
with p.open(newline='', encoding='utf-8') as f:
    rows = list(csv.DictReader(f))
print('Rows:', len(rows))
print('Blocked:', sum(r['evaluation_status'] == 'BLOCKED' for r in rows))
print('Review:', sum(r['evaluation_status'] == 'REVIEW_REQUIRED' for r in rows))
print('Scored:', sum(bool(r['weighted_score']) for r in rows))
```

```
print('QUO001:', next(r['evaluation_status'] for r in rows if r['quotation_id'] ==
'QUO001'))
assert all(not r['weighted_score'] for r in rows if r['evaluation_status'] !=
'ELIGIBLE_FOR_ANALYSIS')
assert all(r['recommendation_status'] != 'APPROVED' for r in rows)
PY
```

Expected learning: An eligibility gate and a scoring function solve different problems. The gate protects business rules; the score compares remaining options.

8. Remaining policy decisions

The 100 inspected units / 10 completed deliveries evidence thresholds, EUR landed-cost definition, 12-month history window, and tie-break rules remain configurable proposals until approved. This prototype inherits upstream results and uses cost/lead/quotation-ID sorting for stable display of equal scores; this should not be confused with an approved purchasing policy.

9. Next checkpoint

Checkpoint 3.1 — Automated testing and architecture refactor. Introduce unit tests, separate data-loading and evaluation logic, and test expired quotes, missing compliance, low-history suppliers, and ties. A production-ready design will also need point-in-time snapshots and a complete audit trail.